

Digital Employee Platform

System Architecture & Data Flow — Full Reference

منصة الموظف الرقمي

شرح تفصيلي للـ System Architecture والـ Data Flow

Contents: (1) System Architecture & Data Flow — English · (2) يبرعلاب حرشل اسفن (3) Viva Q&A

PART 1 — System Architecture & Data Flow (English)

1. High-Level Overview

The system is split into three main parts: the Client Layer, the Backend, and the AI Engine. All three communicate with each other through clearly defined channels — REST/HTTPS, WebRTC, and JWT-secured requests — so that each part can be built, deployed, and scaled independently.

2. Client Layer

There are three types of clients, each talking to the backend differently:

- Customer Chat Widget — an embedded widget on the business's website. The customer types orders, questions, or support requests, and it talks to the Backend over REST/HTTPS.
- Customer/Meeting Device — used for voice calls (phone or in-room mic). It talks directly to the Voice AI Engine via WebRTC — it does not pass through the Backend at all.
- Owner/Admin Frontend App — the dashboard used to manage orders, tickets, menu, and analytics. It talks to the Backend using JWT-authenticated requests.

3. Backend — .NET 9 Web API (Clean Architecture)

The Backend is the heart of the system, built with Clean Architecture across four layers:

- Presentation Layer — the entry point: Controllers for Auth, Business, Orders, Tickets, Dashboard, etc., wrapped with JWT Middleware and Error Middleware.
- Service Layer — the brain: Services, DTOs, and FluentValidation Validators. All business logic and orchestration happens here.
- Domain Layer — the core: Entities (Business, User, Order, Ticket, Interaction), Repository Interfaces, and Enums — with zero external dependencies.
- Data Access Layer (DAL) — talks to the database: EF Core Repositories, Unit of Work, ApplicationDbContext, and Migrations.

4. Infrastructure Layer (Cross-Cutting)

Sitting alongside the Backend and serving all layers, Infrastructure handles JWT token issuing, Google OAuth verification, Email notifications, and holds the AI service URLs and configuration.

5. Database

The DAL talks to SQL Server — DigitalEmployeeDB — which holds all tables: Users, Business, Orders, Tickets, Menu, Knowledge Base, Interactions, Messages, and Audit Logs.

6. AI Engine — Independent Service

The AI Engine is a completely separate server, split into two parts:

- Chat AI Service — receives messages from the Backend, performs intent detection, generates replies, and keeps a per-business knowledge-base cache. The Backend calls it via REST: **POST**

/api/v1/chat.

- Voice AI Engine — the direction is reversed here: it calls the Backend, not the other way around. It performs STT → LLM reasoning → TTS, pushes conversation turns and analysis to the Backend, and reports escalations.

7. Google OAuth & Orchestration

Owners and Admins log in via Google OAuth 2.0; the Infrastructure Layer verifies the Google ID token. In the chat flow, the Backend is the orchestrator — it calls the AI and decides whether to create an order or a ticket based on the flags the AI returns. In the voice flow it is the opposite: the AI Engine drives the conversation and reports back to the Backend.

8. Design Patterns Used

- Repository Pattern — isolates data access. Instead of repeating CRUD logic everywhere, it's written once in **IRepository<T>**.
- Unit of Work — aggregates all Repositories and ensures **SaveChanges** runs inside a single atomic transaction.
- DTO Pattern — Domain entities are never returned to the client directly; each entity has a DTO defining exactly what the client sees.
- FluentValidation — every DTO has a Validator that runs automatically before the request reaches the Controller.

9. Data Flow Diagram — Level 2 (Decomposition of Process 2.0, Backend API)

This diagram decomposes the Backend (Process 2.0 from Level 1) into six internal sub-processes, showing exactly how data moves between the Frontend, these sub-processes, the database, and the AI Engine.

9.1 The Six Sub-Processes

- 2.1 Auth & User Management — login, JWT issuing, Google OAuth verification, roles. (AuthController · AuthService)
- 2.2 Business Profile & Menu/KB Management — business profile, menu, FAQs, knowledge base, settings, working hours. (Business/MenuItem/KB Controllers)
- 2.3 Order Management — create/update orders, cart totals, recommendations. (OrderController · OrderService)
- 2.4 Ticket & Support Management — tickets, feedback, escalations, notifications.
- 2.5 Conversation Orchestration — relays chat messages to the AI, manages voice session context/turns/analysis. (CustomerChat/Conversation Controllers)
- 2.6 Dashboard & Analytics — aggregates KPIs, sentiment, audit log, reports. (DashboardController · ReportService)

9.2 Frontend ↔ Backend Flows

- Flow 1: Frontend ↔ 2.1 — login request / JWT response.
- Flow 2: Frontend ↔ 2.2 — manage business profile, menu, FAQs/KB, settings, hours.
- Flow 3: Frontend ↔ 2.3 — create/view orders, update status.
- Flow 4: Frontend ↔ 2.4 — raise/view tickets, feedback, notifications.
- Flow 5: Frontend ↔ 2.5 — relay chat message, request voice session token.
- Flow 6: Frontend ↔ 2.6 — load dashboard summary, KPIs, recent audit logs.

9.3 Backend ↔ AI Engine Flows

- Flow 9 (2.5 ↔ AI Engine, dashed) — per-message chat call; voice context pull / turn push / session end / escalate. This is the only live, synchronous connection to the AI.
- Flow 8 (AI Engine ↔ 2.2, dashed) — knowledge-base sync, a one-way push triggered whenever menu/FAQ/KB content changes.

Note: 2.6 Dashboard & Analytics has no connection to the AI Engine at all. It only reads from the database to build reports — it never calls, and is never called by, the AI Engine.

9.4 Sub-Process ↔ Database (D1) Ownership

- 2.1 ↔ D1 — reads/writes Users (ASP.NET Identity).
- 2.2 ↔ D1 — Business, MenuCategory, MenuItem, KnowledgeBase, Setting, WorkingHours.
- 2.3 ↔ D1 — Order, OrderItem.
- 2.4 ↔ D1 — Ticket, Feedback, Notification.
- 2.5 ↔ D1 — Interaction, Message, InteractionAnalysis, CallSummary, Sentiment.
- 2.6 ↔ D1 — read-only aggregation across every table + AuditLog.

9.5 Key Notes

- All six sub-processes share the single D1 data store — each owns a distinct slice of the schema; only 2.6 is read-only.
- 2.5 Conversation Orchestration is the only sub-process that talks to the AI Engine (per the AI ↔ Backend Contract V2); 2.2 separately pushes KB-sync updates.
- This is the decomposition of Process 2.0 from DFD Level 1 — see Level 1 for how 2.0 fits into the whole platform.

شرح الـ System Architecture والـ Data Flow (عربي)

الجزء الثاني

١. نظرة عامة على النظام

وWebRTC، وrequests متأمنة بالـ JWT — عشان كل جزء يقدر يتبني ويتنشر ويتوسع لوحده من غير ما يآثر على الباقي. رئيسية: الـ Client Layer، والـ Backend، والـ AI Engine. الثلاثة بيتكلموا مع بعض بطرق محددة — REST/HTTPS، النظام مقسم لثلاث أجزاء

٢. الـ Client Layer

عندنا 3 أنواع clients، وكل واحد بيتكلم مع الـ Backend بطريقة مختلفة:

- أو أسئلة أو orders بيكتب فيه الـ customer، business مدمج في موقع الـ widget — Customer Chat Widget
- REST/HTTPS عن طريق الـ Backend وبيتكلم مع الـ support requests.
- Customer/Meeting Device — Voice AI (تليفون أو مايك في الأوضة)، وده بيتكلم مباشرة مع الـ voice calls للـ Customer/Meeting Device
- Backend مش بيعدي على الـ WebRTC عن طريق الـ Engine
- menu والـ tickets والـ orders الـ owner اللي بيدير بيه الـ dashboard الـ Owner/Admin Frontend App
- JWT عن طريق الـ Backend وبيتكلم مع الـ analytics والـ

٣. الـ .NET 9 Web API Backend — (بنمط Clean Architecture)

الـ Backend هو قلب النظام، اتبنى بأربع layers:

- والـ Tickets والـ Orders والـ Business والـ Auth بتاعت الـ Controllers: نقطة الدخول — Presentation Layer
- Error Middleware وJWT Middleware وعليها Dashboard.
- business كل الـ — FluentValidation Validators وDTOs والـ Services العقل: فيه الـ Service Layer
- orchestration والـ logic
- Repository والـ Interaction وTicket وOrder وUser وBusiness زي الـ Entities النواة: الـ Domain Layer
- dependency من غير أي — Enums والـ Interfaces
- Data Access Layer (DAL) — EF Core Repositories، Unit of Work،
- Migrations، والـ ApplicationDbContext

٤. الـ (Cross-Cutting Infrastructure Layer)

JWT، بتتحقق من الـ Google OAuth، بتبعت الـ Email notifications، ومنها بيبجي الـ AI service URLs والـ config. موجودة جنب الـ Backend وبتخدم كل الـ layers: بتعمل token issuing

٥. الداتابيز

Users، Business، Orders، Tickets، Menu، Knowledge Base، Interactions، Messages، Audit Logs الـ DAL بيتكلم مع الـ DigitalEmployeeDB — SQL Server — اللي فيها كل الجداول:

٦. الـ AI Engine — سيرفس منفصل خالص

الـ AI Engine سيرفر منفصل تماما، وعنده جزئين:

- وعنده، reply ويولد الـ intent detection يعمل، Backend، من الـ message بيستقبل الـ Chat AI Service — الـ REST: POST /api/v1/chat. بيكلمه عن طريق الـ Backend الـ business لكل الـ knowledge-base الـ cache الـ
- الـ LLM reasoning ثم الـ STT مش العكس. يعمل Backend الاتجاه هنا معكوس: هو اللي بيكلم الـ Voice AI Engine — الـ escalations. ويبيلغ عن الـ Backend، الـ analysis والـ conversation turns الـ push ويب، الـ TTS.

٧. الـ Google OAuth والـ Orchestration

- الـ flags اللي الـ AI بيرجعها. أما في الـ voice flow فالعكس صحيح: الـ AI Engine هو اللي يقود المحادثة ويبيلغ الـ Backend.
- في الـ chat flow، الـ Backend هو الـ orchestrator — هو اللي بيكلم الـ AI ويقرر ينشئ order أو ticket بناء على
- Admin بيعملوا login عن طريق الـ Google OAuth 2.0، والـ Infrastructure Layer بتتحقق من الـ Google ID token. الـ Owner والـ

٨. الـ Design Patterns المستخدمة

- في كل مكان، اتكتب مرة واحدة في الـ CRUD بدل ما نكرر الـ data access. بيعزل الـ Repository Pattern — الـ IRepository<T>.
- واحدة transaction بتتعمل جوه الـ SaveChanges ويضمن إن الـ Repositories بيجمع كل الـ Unit of Work — الـ atomic.
- بيحدد بالطبط إيه اللي الـ DTO عندها entity مباشرة، كل client مش بترجع الـ Domain entities الـ DTO Pattern — الـ client يشوفه.
- الـ Controller يوصل للـ request بيشتغل أوتوماتيك قبل ما الـ Validator عنده الـ DTO كل الـ FluentValidation.

٩. Data Flow Diagram — Level 2 (Backend الـ Process 2.0، تفصيل الـ) API)

داخلية، وبيوضح بالظبط إزاي الداتا بتتحرك بين الـ Frontend، والـ sub-processes دي، والداتا بيز، والـ AI Engine. الديجرام ده بي فصل الـ Backend (الـ Process 2.0 من الـ Level 1) لستة الـ sub-processes

٩.١ الستة Sub-Processes

- roles. والـ Google OAuth، التحقق من، JWT إصدار الـ login، الـ 2.1 Auth & User Management —
- الـ FAQs، المنيو، الـ business، بروفايل الـ 2.2 Business Profile & Menu/KB Management — الإعدادات، ساعات العمل، knowledge base.
- recommendations. الـ cart، إجمالي الـ orders إنشاء/تعديل الـ 2.3 Order Management —
- escalations، الـ feedback، الـ tickets الـ 2.4 Ticket & Support Management —
- voice session وبيدير الـ AI، الـ chat بيوصل رسائل الـ 2.5 Conversation Orchestration — (context/turns/analysis).
- audit log، الـ sentiment، الـ KPIs، بيجمع الـ 2.6 Dashboard & Analytics —

٩.٢ تدفقات الـ Frontend ↔ Backend

- JWT رد بالـ / login طلب — 2.1 ↔ Frontend Flow 1:
- الإعدادات وساعات العمل FAQs/KB والمنيو والـ business إدارة بروفايل الـ 2.2 ↔ Frontend Flow 2:
- تحديث الحالة، orders إنشاء/عرض الـ 2.3 ↔ Frontend Flow 3:
- الإشعارات، feedback، الـ tickets فتح/عرض الـ 2.4 ↔ Frontend Flow 4:
- voice. لجلسة token طلب، chat تمرير رسالة الـ 2.5 ↔ Frontend Flow 5:
- audit logs. آخر الـ KPIs، الـ dashboard تحميل ملخص الـ 2.6 ↔ Frontend Flow 6:

٩.٣ تدفقات الـ Backend ↔ AI Engine

- إنهاء الـ / turns دفع الـ / voice الـ context، سحب chat منقطع) — نداء لكل رسالة، 2.5 ↔ AI Engine Flow 9
- AI. ده الاتصال اللايف المتزامن الوحيد مع الـ escalation. الـ session /
- يحصل كل ما محتوى المنيو أو الـ push وده، knowledge base منقطع) — مزامنة الـ 2.2 ↔ AI Engine Flow 8
- يتغير KB أو الـ FAQ في اتجاه واحد
- بالـ AI Engine خالص. هي بس بتقرا من الداتا بيز عشان تبني التقارير — مش بتنادي، ومش بينادي عليها من، الـ AI Engine.
- ملحوظة: 2.6 Dashboard & Analytics مالهش أي اتصال

٩.٤ ملكية كل Sub-Process لجزء من الداتا بيز (D1)

- Users (ASP.NET Identity) قراءة/كتابة جدول — D1 ↔ 2.1
- Business، MenuCategory، MenuItem، KnowledgeBase، Setting، WorkingHours. — D1 ↔ 2.2
- Order، OrderItem. — D1 ↔ 2.3
- Ticket، Feedback، Notification. — D1 ↔ 2.4

• 2.5 ↔ D1 — Interaction، Message، InteractionAnalysis، CallSummary، Sentiment.

• 2.6 ↔ D1 — AuditLog + عبر كل الجداول (read-only) قراءة فقط

٩.٥ ملاحظات مهمة

كلهم يشاركون نفس الـ data store D1 — كل واحد يمتلك جزء مختلف من الـ schema؛ 2.6 بس اللي الـ read-only.

• الستة sub-processes

↔ AI (حسب) AI Engine الوحيد اللي بيتكلم مع الـ sub-process هو الـ 2.5 Conversation Orchestration

• KB. ؛ 2.2 بتتبع بشكل منفصل تحديثات مزامنة الـ (Backend Contract V2)

• ده تفصيل للـ Process 2.0 من DFD Level 1 — ارجع للـ Level 1 عشان تشوف إزاي 2.0 بتندمج في المنصة كلها.

PART 3 — Viva Questions & Answers

Why does only sub-process 2.5 (Conversation Orchestration) talk to the AI Engine, and not the others?

Because conversation handling (chat and voice) is the only feature that needs live AI interaction. Isolating it in one sub-process means the AI integration contract (request/response shape, session handling, escalation logic) lives in a single place. If Order or Ticket Management called the AI directly, that integration logic would be duplicated, and any change to the AI contract would ripple across multiple controllers instead of one.

Why is 2.6 (Dashboard & Analytics) read-only?

Its only job is aggregation and reporting — it doesn't own any business data, it just reads across every table (orders, tickets, interactions, etc.) plus the audit log to compute KPIs and sentiment summaries. Making it read-only enforces single ownership of the schema: every other sub-process is the sole writer for its own tables, so there's no risk of two sub-processes racing to update the same row.

What's the difference between Flow 8 and Flow 9 (the two AI Engine connections)?

Flow 9 is synchronous, per-interaction traffic — a chat message going out and a reply coming back, or voice session context/turns/escalations. Flow 8 is a one-way push, triggered only when a business updates its menu, FAQs, or knowledge base — it keeps the AI's per-business knowledge cache in sync. Different triggers, different directions — modeled as two separate flows even though both connect to the same external AI Engine.

Why split Auth into its own sub-process instead of folding it into another one?

Auth is a cross-cutting concern every other sub-process depends on indirectly (via JWT), but it has its own distinct data (Users, roles) and its own external dependency (Google OAuth). Isolating it keeps JWT-issuing and identity-verification logic testable and swappable without touching business logic elsewhere.

Does the Frontend ever bypass the Backend to reach the AI Engine directly?

Not in the chat/dashboard path — the Frontend always goes through the Backend there. The voice device path (WebRTC directly to the Voice AI Engine) is a separate flow shown at DFD Level 1, since this Level 2 diagram is specifically the decomposition of the Backend API process.

Why is the database shown as a single data store (D1) instead of splitting it per sub-process?

Physically it's one SQL Server database — splitting it into multiple data stores in the diagram would misrepresent the real architecture. Ownership boundaries (who reads/writes what) are captured through the labeled flows instead, which is the correct way to show logical separation within a single physical store in a DFD.

Why use Clean Architecture with four layers instead of a simpler layered setup?

Clean Architecture keeps the Domain layer free of external dependencies (no EF Core, no ASP.NET references), so business rules can be tested in isolation and the persistence or web framework could be swapped without touching core logic. The Service layer mediates between the web-facing Presentation layer and the Domain/DAL, keeping orchestration logic out of both Controllers and Entities.

Why is the AI Engine a separate service instead of a module inside the .NET Backend?

The AI Engine has a completely different runtime and scaling profile (LLM inference, STT/TTS, GPU-bound workloads) compared to the Backend's request/response API workload. Keeping it separate lets each service scale, deploy, and fail independently, and keeps AI-specific dependencies out of the .NET codebase entirely.

Why JWT instead of cookie-based sessions?

JWT is stateless — the Backend doesn't need to store session state server-side, which fits a system with multiple client types (chat widget, admin dashboard) hitting the same API. It also travels naturally in an Authorization header, which works cleanly for both browser and non-browser clients.

What does the Unit of Work actually solve that individual Repositories don't?

A single business operation often touches multiple tables at once — e.g., creating an order might update Order, OrderItem, and trigger a Notification. Without Unit of Work, each Repository would call SaveChanges independently, risking a partial write if one save succeeds and another fails. Unit of Work wraps all of them in one transaction, so the operation is atomic — all changes commit together or none do.